

=====

GITBRAIN – AI-POWERED GITHUB REPOSITORY INTELLIGENCE ASSISTANT

PRACTICAL IMPLEMENTATION MENTOR GUIDE

For Students, Interns, Freshers & Self-Learners

=====

Based on: GitBrain Internship Project Report

Tech Stack: Python · FastAPI · LlamaIndex · Llama 3.1 (Groq) · ChromaDB
sentence-transformers · Streamlit · GitHub REST API · GitHub Actions

=====

HOW TO USE THIS GUIDE

Read this file from top to bottom on your first pass.

After that, use each SECTION as a reference while building that specific layer.

Every section tells you WHAT to build, WHY it exists, and HOW to implement it.

Code snippets are illustrative – write your own, don't copy blindly.

Follow the MVP BUILD ORDER in Section 15 if you are just getting started.

=====

SECTION 1: PROJECT IMPLEMENTATION OVERVIEW

=====

WHAT IS GITBRAIN?

GitBrain is a RAG (Retrieval-Augmented Generation) application.

It lets you ask questions about a GitHub repository in plain English and get cited, accurate answers – like having a senior developer who has read the entire codebase explain it to you instantly.

THE THREE-SENTENCE MENTAL MODEL

1. GitBrain reads a GitHub repository and converts its code into searchable "chunks" stored in a local vector database (ChromaDB).
2. When you ask a question, it finds the most relevant code chunks using semantic similarity (not keyword matching).
3. It gives those chunks to Llama 3.1 (running on Groq) and asks it to generate a human-readable answer with source file citations.

THE THREE LAYERS

Layer 1 – INGESTION LAYER

Fetches code from GitHub → splits it into chunks → embeds → stores in ChromaDB

Triggered: manually at start, or automatically via GitHub Actions on PR merge

Layer 2 – INTELLIGENCE LAYER (RAG Core)

Receives a user query → finds relevant chunks → builds a prompt → calls LLM

Returns: generated answer + source file citations

49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96

Layer 3 – INTERFACE LAYER

FastAPI backend exposes REST endpoints (/ingest, /query, /health)
Streamlit frontend provides the chat UI

CORE DATA OBJECTS (burn these into memory)

FileContent = { path, content, language } – raw file from GitHub
Chunk = { text, file_path, chunk_type, – processed code unit
chunk_name, start_line,
end_line, language }
EmbeddedChunk = Chunk + { embedding: [float×384] } – chunk with vector
QueryResult = { answer, sources: [{file, lines}] } – final response

WHAT SUCCESS LOOKS LIKE

You know GitBrain works when you can:
✓ Run the ingester on any public GitHub repo and get chunks stored
✓ Ask "how does authentication work?" and get a cited answer
✓ Ask a question with no relevant code and get "I cannot find information"
✓ Push a change to GitHub, trigger Actions, and see new code discovered

=====

SECTION 2: DEVELOPMENT ROADMAP

=====

TOTAL TIME ESTIMATE: 7-8 weeks for a motivated beginner

WEEK 1-2 | PHASE 1: Foundation

Goal: Get code from GitHub into your terminal as text.

- Tasks:
- Set up Python environment and install all dependencies
 - Get your GitHub personal access token
 - Build github_ingester.py – fetch file tree + file contents
 - Build code_chunker.py – split files into function/class chunks
 - Test: print chunks to terminal for a small repo (10-20 files)

"Done" signal: Running `python test\_ingester.py owner/repo` prints
a list of chunks with file paths and line numbers.

WEEK 3 | PHASE 2: Vector Intelligence

Goal: Store chunks in ChromaDB and retrieve by semantic similarity.

97 Tasks:

- 98 □ Build embedder.py – generate vectors using sentence-transformers
- 99 □ Build vector_store.py – wrap ChromaDB operations
- 100 □ Connect ingester → chunker → embedder → ChromaDB (pipeline script)
- 101 □ Test: query ChromaDB manually and verify relevant chunks are returned

102
103 "Done" signal: Running `python test\_search.py "how does login work"` returns
104 relevant code chunks from ChromaDB sorted by similarity score.

105 106 WEEK 4 | PHASE 3: LLM Integration + FastAPI Backend

107
108 Goal: Build the backend API with working Q&A.

109
110 Tasks:

- 111 □ Build llm_client.py – wrap Groq API
- 112 □ Build rag_engine.py – orchestrate the full query pipeline
- 113 □ Build FastAPI app with /health, /ingest, /query endpoints
- 114 □ Test all endpoints using curl or Postman

115
116 "Done" signal: `curl -X POST localhost:8000/query -d '{"question":"..."}'`
117 returns a JSON response with answer and sources fields.

118 119 WEEK 5 | PHASE 4: Frontend

120
121 Goal: Wrap everything in a usable chat UI.

122
123 Tasks:

- 124 □ Build Streamlit app with sidebar (repo config) + main chat area
- 125 □ Display answers with source citation panel
- 126 □ Add session-level conversation memory
- 127 □ Connect to FastAPI backend via HTTP requests

128
129 "Done" signal: You can demo GitBrain to someone non-technical using only
130 the Streamlit browser interface.

131 132 WEEK 6 | PHASE 5: Automation + Polish

133
134 Goal: Add GitHub Actions re-indexing and source citations.

135
136 Tasks:

- 137 □ Implement incremental re-indexing (only changed files)
- 138 □ Build GitHub Actions workflow YAML
- 139 □ Polish source citation parsing in rag_engine.py
- 140 □ Display citations as clickable links in Streamlit

141
142 "Done" signal: Merge a PR in your test repo and see GitBrain automatically
143 discover the new code without manual re-indexing.

- 146
-
- 147 Tasks:
- 148 □ Write pytest unit tests for chunker, embedder, vector_store
 - 149 □ Write integration tests for /ingest and /query endpoints
 - 150 □ Deploy FastAPI to Railway, Streamlit to Streamlit Cloud
 - 151 □ Write README with setup instructions and demo screenshots
 - 152 □ Finalize internship report

153

154 GOLDEN RULE: Build one vertical slice first.

155 Before adding any new feature, make sure you can:

156 Ingest 1 file → embed 1 chunk → store → retrieve → generate answer → display

157 This end-to-end slice should work before you add more files or features.

158

159

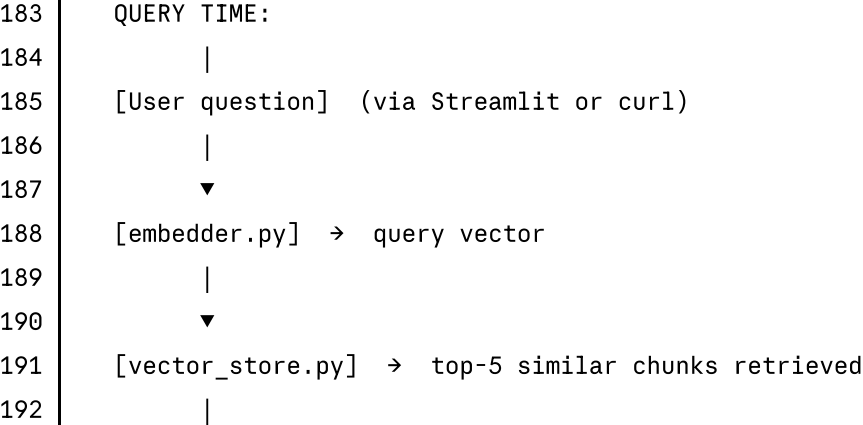
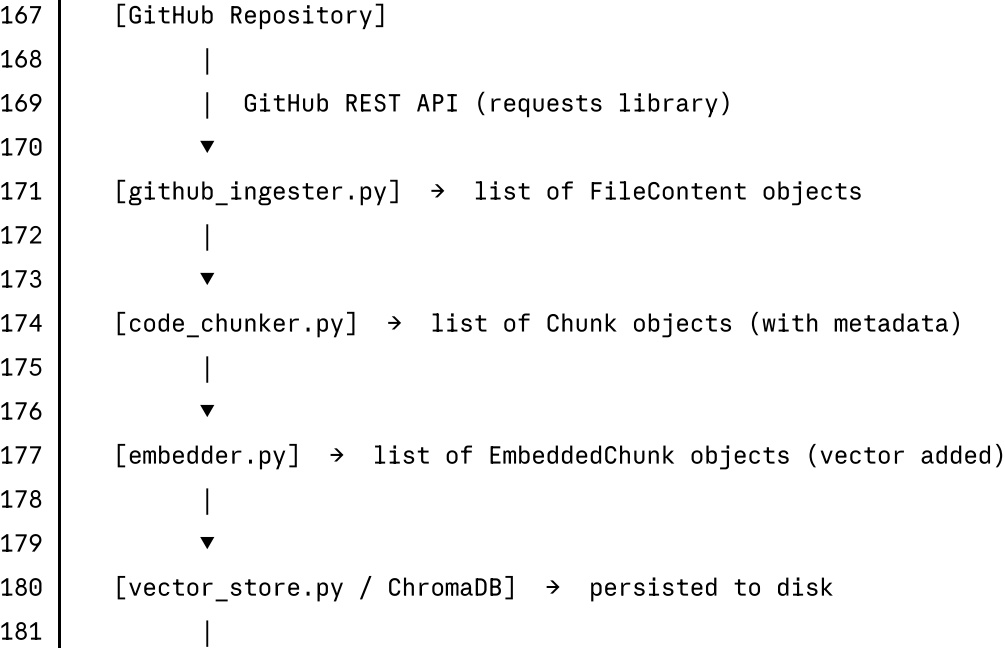
160 =====

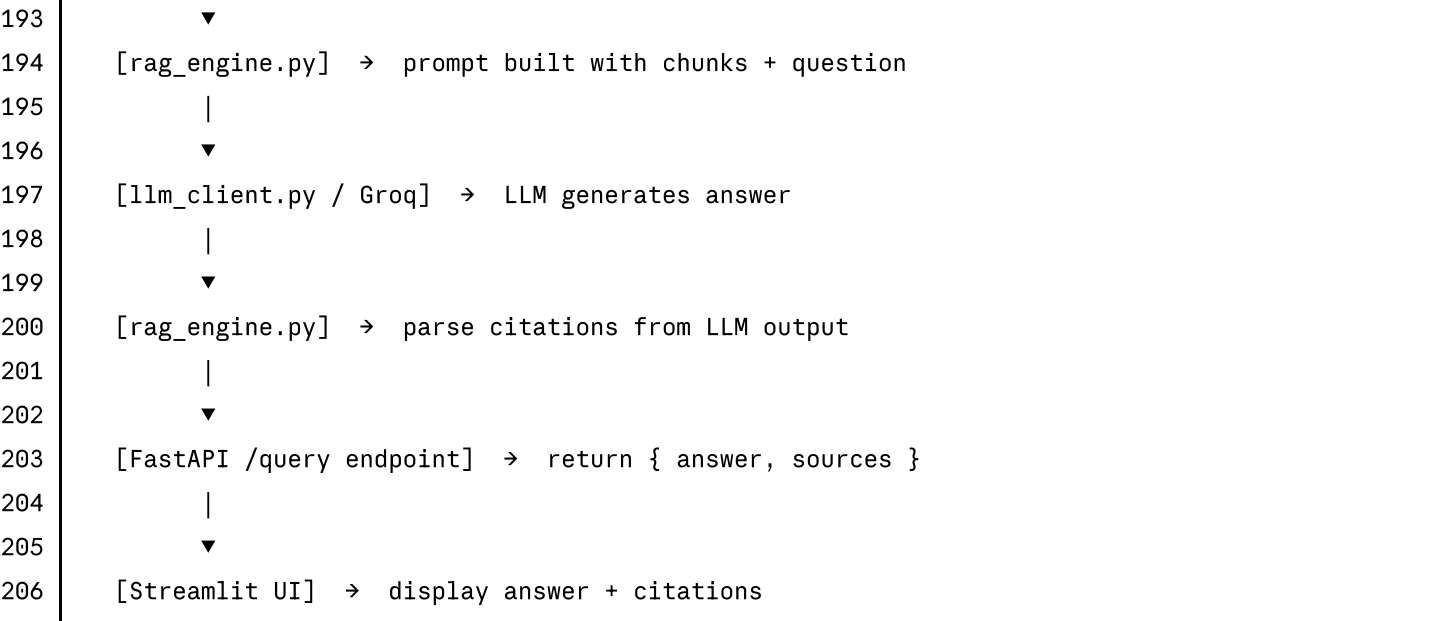
161 SECTION 3: SYSTEM ARCHITECTURE BREAKDOWN

162 =====

163

164 HOW THE LAYERS CONNECT





208 MODULE DEPENDENCY MAP

210	api/routes/ingest.py	depends on →	core/github_ingester.py
211			core/code_chunker.py
212			core/embedder.py
213			core/vector_store.py
214			
215	api/routes/query.py	depends on →	core/rag_engine.py
216			(which depends on embedder, vector_store, llm_client)
217			
218	core/rag_engine.py	depends on →	core/embedder.py
219			core/vector_store.py
220			core/llm_client.py
221			
222	frontend/app.py	depends on →	FastAPI backend (HTTP requests only)
223			(no direct Python imports from core/)
224			

225 KEY INSIGHT: The frontend should NEVER import directly from core/.
226 It communicates with the backend exclusively through HTTP.
227 This lets you develop and test them independently.

230 =====
231 SECTION 4: BACKEND LAYER IMPLEMENTATION GUIDE
232 =====

234 A. PURPOSE

235 _____
236 The backend is the brain of the operation.
237 It receives requests from the frontend, orchestrates the pipeline,
238 and returns structured responses.
239 Built with FastAPI – Python's modern, async, type-safe API framework.

B. RESPONSIBILITIES

- Accept and validate incoming HTTP requests
- Trigger ingestion pipeline when /ingest is called
- Trigger query pipeline when /query is called
- Return structured JSON responses
- Handle errors gracefully without crashing
- Log what is happening at each step

C. CORE FILES

api/main.py → FastAPI app creation and router registration
api/routes/health.py → GET /health endpoint
api/routes/ingest.py → POST /ingest endpoint
api/routes/query.py → POST /query endpoint
config/settings.py → All config loaded from environment variables

D. BUILD ORDER (follow this exactly)

Step 1: Create api/main.py with basic FastAPI app (no endpoints yet)
Step 2: Add GET /health endpoint – make sure it returns {"status": "ok"}
Step 3: Create config/settings.py with Pydantic BaseSettings
Step 4: Add POST /ingest endpoint (stub first, wire pipeline later)
Step 5: Add POST /query endpoint (stub first, wire pipeline later)
Step 6: Wire the actual pipeline logic into the endpoints
Step 7: Add error handling and logging

Do NOT skip ahead to Step 6 without testing Steps 1-5 first.

E. WHAT api/main.py LOOKS LIKE

```
from fastapi import FastAPI
from api.routes import ingest, query, health

app = FastAPI(title="GitBrain", version="1.0.0")
app.include_router(health.router)
app.include_router(ingest.router)
app.include_router(query.router)

# Run with: uvicorn api.main:app --reload --port 8000
```

F. WHAT config/settings.py LOOKS LIKE

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    github_token: str = ""
    groq_api_key: str = ""
```

```

289     chroma_persist_dir: str = "./chroma_db"
290     embedding_model: str = "all-MiniLM-L6-v2"
291     llm_model: str = "llama3-8b-8192"
292     top_k_results: int = 5
293     similarity_threshold: float = 0.4
294
295     class Config:
296         env_file = ".env"
297
298 settings = Settings() # import this singleton everywhere
299
300 G. WHAT THE /ingest ENDPOINT LOOKS LIKE
301 

---


302 from fastapi import APIRouter
303 from pydantic import BaseModel
304 from core.github_ingester import ingest_repository
305
306 router = APIRouter()
307
308 class IngestRequest(BaseModel):
309     repo_url: str
310     token: str = ""
311     incremental: bool = False
312
313 class IngestResponse(BaseModel):
314     status: str
315     chunks_indexed: int
316
317 @router.post("/ingest", response_model=IngestResponse)
318 async def ingest(request: IngestRequest):
319     # Call the pipeline – returns number of chunks indexed
320     count = ingest_repository(request.repo_url, request.token, request.incremental)
321     return IngestResponse(status="success", chunks_indexed=count)
322
323 H. WHAT THE /query ENDPOINT LOOKS LIKE
324 

---


325 class QueryRequest(BaseModel):
326     question: str
327     repo_name: str
328     top_k: int = 5
329
330 class Source(BaseModel):
331     file: str
332     lines: str
333
334 class QueryResponse(BaseModel):
335     answer: str
336     sources: list[Source]

```

```
337
338 @router.post("/query", response_model=QueryResponse)
339 async def query(request: QueryRequest):
340     result = rag_engine.query(request.question, request.repo_name, request.top_k)
341     return QueryResponse(answer=result["answer"], sources=result["sources"])
342
```

343 I. ERROR HANDLING PATTERN (use in every endpoint)

```
344
345 from fastapi import HTTPException
346
347 try:
348     result = some_pipeline_call()
349 except GitHubRateLimitError as e:
350     raise HTTPException(status_code=429, detail=str(e))
351 except RepoNotFoundError as e:
352     raise HTTPException(status_code=404, detail=str(e))
353 except Exception as e:
354     logger.error(f"Unexpected error: {e}")
355     raise HTTPException(status_code=500, detail="Internal server error")
356
```

357 J. LOGGING SETUP (put in api/main.py)

```
358
359 import logging
360 logging.basicConfig(
361     level=logging.DEBUG, # Change to INFO for production
362     format="%(asctime)s | %(name)s | %(levelname)s | %(message)s"
363 )
364 logger = logging.getLogger(__name__)
365
```

366 K. HOW TO TEST THE BACKEND

368 Start server: `uvicorn api.main:app --reload --port 8000`

370 Test health:

```
371 curl http://localhost:8000/health
372
```

373 Test ingest:

```
374 curl -X POST http://localhost:8000/ingest \
375     -H "Content-Type: application/json" \
376     -d '{"repo_url": "https://github.com/tiangolo/fastapi"}'
377
```

378 Test query:

```
379 curl -X POST http://localhost:8000/query \
380     -H "Content-Type: application/json" \
381     -d '{"question": "how does routing work?", "repo_name": "fastapi"}'
382
```

383 L. COMMON BACKEND MISTAKES

384


```
385     × Putting business logic directly in the route function
386     → Always separate into a service/core module
387
388     × Not using Pydantic models for request/response
389     → Always define BaseModel classes for all inputs and outputs
390
391     × Forgetting to handle async properly
392     → If calling sync libraries (sentence-transformers, ChromaDB) from async,
393         consider running in a thread pool: await asyncio.get_event_loop().run_in_executor(None, sync_f
unc)
394
395     × Hardcoding config values inside route functions
396     → Always import from config/settings.py
397
```

398 M. VALIDATION CHECKLIST

```
399
400     □ GET /health returns {"status": "ok"} with HTTP 200
401     □ POST /ingest with valid repo returns {"status": "success", "chunks_indexed": N}
402     □ POST /ingest with invalid repo returns HTTP 404
403     □ POST /query with good question returns {"answer": "...", "sources": [...]}
404     □ POST /query for unindexed repo returns HTTP 404 or appropriate error
405     □ Server doesn't crash on bad input – returns HTTP 422 from Pydantic validation
406
407
```

408 =====

409 SECTION 5: FRONTEND LAYER IMPLEMENTATION GUIDE

410 =====

412 A. PURPOSE

```
413
414     The Streamlit frontend is what users actually interact with.
415     It wraps the backend API in a browser-based chat interface.
416     Streamlit is chosen because it lets you build UIs in pure Python
417     with minimal code – ideal for an internship project timeline.
418
```

419 B. RESPONSIBILITIES

```
420
421     - Accept repository URL from user and trigger ingestion
422     - Accept natural language questions and display answers
423     - Display source citations alongside each answer
424     - Maintain conversation history within a session
425     - Show loading indicators while backend is processing
426     - Handle and display errors without crashing
427
```

428 C. CORE FILE

```
429
430     frontend/app.py → Single Streamlit file for the entire UI
431
```

D. UI LAYOUT (two regions)

SIDEBAR (left panel)

- Text input: "GitHub Repository URL"
- Text input: "GitHub Token (optional)"
- Button: "Ingest Repository"
- Status display: "✓ Indexed: owner/repo – 342 chunks"
- Expander: "File Tree" (list of files in indexed repo)

MAIN AREA (right panel)

- Title: "GitBrain – Ask your codebase anything"
- Chat message history (scrollable)
- For each message pair:
 - User message bubble
 - GitBrain answer bubble
 - Expandable "Sources" section below the answer
- Text input at bottom: "Ask a question..."
- Button: "Send" or just hit Enter

E. BUILD ORDER

- Step 1: Build sidebar with repo URL input and ingest button
- Step 2: Call backend /ingest from sidebar and show success/error
- Step 3: Build basic question input and send button in main area
- Step 4: Call backend /query and display raw answer as text
- Step 5: Add chat history using st.session_state
- Step 6: Style answer display with source citations panel
- Step 7: Add loading spinners and error messages
- Step 8: Add file tree display in sidebar expander

F. SESSION STATE (how Streamlit remembers things)

Streamlit re-runs the entire script on every interaction.

Use st.session_state to persist data between runs.

Initialize at the top of app.py

if "messages" not in st.session_state:

 st.session_state.messages = [] # list of {role, content, sources}

if "repo_name" not in st.session_state:

 st.session_state.repo_name = None # currently indexed repo

G. CALLING THE BACKEND

import requests

BACKEND_URL = "http://localhost:8000" # or read from st.secrets

```

480 def call_ingest(repo_url, token=""):
481     try:
482         response = requests.post(
483             f"{BACKEND_URL}/ingest",
484             json={"repo_url": repo_url, "token": token},
485             timeout=300 # ingestion can take several minutes
486         )
487         response.raise_for_status()
488         return response.json()
489     except requests.exceptions.Timeout:
490         return {"error": "Ingestion timed out – repo may be too large"}
491     except requests.exceptions.RequestException as e:
492         return {"error": str(e)}
493
494 def call_query(question, repo_name, top_k=5):
495     try:
496         response = requests.post(
497             f"{BACKEND_URL}/query",
498             json={"question": question, "repo_name": repo_name, "top_k": top_k},
499             timeout=30
500         )
501         response.raise_for_status()
502         return response.json()
503     except requests.exceptions.RequestException as e:
504         return {"error": str(e)}
505
506 H. DISPLAYING CHAT MESSAGES
507 

---


508 # Render message history
509 for msg in st.session_state.messages:
510     with st.chat_message(msg["role"]):
511         st.markdown(msg["content"])
512         if msg.get("sources"):
513             with st.expander("Sources"):
514                 for s in msg["sources"]:
515                     st.code(f"{s['file']} (Lines {s['lines']})", language="")
516
517 # Handle new input
518 if prompt := st.chat_input("Ask about the codebase..."):
519     st.session_state.messages.append({"role": "user", "content": prompt})
520     with st.spinner("GitBrain is thinking..."):
521         result = call_query(prompt, st.session_state.repo_name)
522     if "error" in result:
523         answer = f"Error: {result['error']}"
524         sources = []
525     else:
526         answer = result["answer"]
527         sources = result["sources"]

```

```
st.session_state.messages.append({
    "role": "assistant", "content": answer, "sources": sources
})
st.rerun()
```

I. INGEST BUTTON IN SIDEBAR

```
with st.sidebar:
    st.header("Repository Setup")
    repo_url = st.text_input("GitHub Repository URL",
                             placeholder="https://github.com/owner/repo")
    token = st.text_input("GitHub Token (optional)", type="password")
    if st.button("Ingest Repository"):
        with st.spinner("Ingesting repository... this may take a few minutes"):
            result = call_ingest(repo_url, token)
            if "error" in result:
                st.error(result["error"])
            else:
                repo_name = repo_url.rstrip("/").split("/")[-2] + "__" + repo_url.rstrip("/").split
                ("/")[-1]
                st.session_state.repo_name = repo_name
                st.success(f"Indexed {result['chunks_indexed']} chunks")
```

J. COMMON FRONTEND MISTAKES

- × Forgetting to initialize session_state keys before using them
 - Always check "if key not in st.session_state" at the top
- × Calling st.rerun() inside a button click without appending to messages first
 - Append all messages to session_state BEFORE calling st.rerun()
- × Setting a very short timeout for ingestion requests
 - Ingestion can take 3-10 minutes for large repos; use timeout=300+
- × Hardcoding the backend URL
 - Use st.secrets["BACKEND_URL"] or an environment variable

K. VALIDATION CHECKLIST

- ☐ Entering a repo URL and clicking Ingest shows a spinner and success message
- ☐ Asking a question shows the answer in a chat bubble
- ☐ Sources appear in a collapsible expander below each answer
- ☐ Previous messages stay visible as you ask more questions
- ☐ An error from the backend shows an error message (not a Python traceback)
- ☐ The page doesn't crash when the backend is offline

=====

SECTION 6: AI / RAG LAYER IMPLEMENTATION GUIDE

A. PURPOSE

This is the intelligence core of GitBrain.

It transforms a user's question into a relevant, cited answer by finding the right code and asking the LLM to explain it.

B. RESPONSIBILITIES

- Embed the user query into a vector
- Search ChromaDB for the most relevant code chunks
- Filter out low-quality matches below similarity threshold
- Build a structured prompt with retrieved code + question
- Call Llama 3.1 via Groq and get the response
- Parse source citations from the LLM output
- Return structured answer with sources

C. CORE FILE: core/rag_engine.py

This is the most important file in the project.

It orchestrates everything. Keep it clean and well-commented.

D. THE FULL QUERY FUNCTION

```
def query(question: str, repo_name: str, top_k: int = 5) -> dict:
    # Step 1: Embed the question
    query_vector = embedder.embed_single(question)

    # Step 2: Retrieve top-k chunks from ChromaDB
    raw_results = vector_store.similarity_search(
        collection_name=repo_name,
        query_vector=query_vector,
        top_k=top_k
    )

    # Step 3: Filter by similarity threshold
    THRESHOLD = settings.similarity_threshold
    filtered = [r for r in raw_results if r["score"] >= THRESHOLD]

    if not filtered:
        return {
            "answer": "I cannot find information about this in the repository.",
            "sources": []
        }

    # Step 4: Build the prompt
```

```

623     prompt = build_prompt(question, filtered)
624
625     # Step 5: Call the LLM
626     llm_response = llm_client.generate(prompt)
627
628     # Step 6: Parse citations
629     sources = parse_citations(llm_response, filtered)
630
631     return {"answer": llm_response, "sources": sources}
632

```

633 E. THE PROMPT TEMPLATE

634
635 This is the most important piece of prompt engineering.
636 Write it carefully. Test it. Iterate on it.

637
638 SYSTEM_PROMPT = """You are GitBrain, an expert code assistant. \
639 You help developers understand GitHub repositories.

640
641 Rules:

- 642 1. Answer the user's question using ONLY the code context provided below.
- 643 2. For each claim in your answer, cite the source file as [FILE: path/to/file.py, Lines X-Y].
- 644 3. If the answer cannot be found in the provided context, say exactly:
645 "I cannot find information about this in the repository."
- 646 4. Do not invent code, functions, or behavior that is not in the context.
- 647 5. Be concise but complete. Use bullet points for multi-step explanations."""

```

648
649 def build_prompt(question: str, chunks: list) -> str:
650     context_blocks = []
651     for i, chunk in enumerate(chunks):
652         block = f"""--- Source {i+1}: {chunk['metadata']['file_path']} \  

653 (Lines {chunk['metadata']['start_line']}-{chunk['metadata']['end_line']}) ---  

654 {chunk['text']}"""
655         context_blocks.append(block)
656
657     context = "\n\n".join(context_blocks)
658
659     return f"""{SYSTEM_PROMPT}

```

660
661 CODE CONTEXT:

662 {context}

663
664 USER QUESTION: {question}

665
666 ANSWER: """

668 F. CITATION PARSING

669
670 The LLM is prompted to write [FILE: path, Lines X-Y] in its answers.

```

671 Parse these markers to show structured citations in the UI.
672
673 import re
674
675 def parse_citations(llm_output: str, retrieved_chunks: list) -> list:
676     # Find all [FILE: ...] patterns in the LLM output
677     pattern = r'\[FILE:\s*(.+?),\s*Lines?\s*(\d+[\--]\d+)\]'
678     matches = re.findall(pattern, llm_output)
679     seen = set()
680     sources = []
681     for file_path, line_range in matches:
682         key = f"{file_path}:{line_range}"
683         if key not in seen:
684             sources.append({"file": file_path.strip(), "lines": line_range})
685             seen.add(key)
686
687     # Fallback: if LLM didn't cite properly, use retrieved chunk metadata
688     if not sources:
689         for chunk in retrieved_chunks:
690             sources.append({
691                 "file": chunk["metadata"]["file_path"],
692                 "lines": f"{chunk['metadata']['start_line']}-{chunk['metadata']['end_line']}"
693             })
694     return sources
695
696 G. LLM CLIENT: core/llm_client.py
697
698 from groq import Groq
699 from config.settings import settings
700
701 client = Groq(api_key=settings.groq_api_key)
702
703 def generate(prompt: str) -> str:
704     for attempt in range(3): # retry up to 3 times
705         try:
706             response = client.chat.completions.create(
707                 model=settings.llm_model, # "llama3-8b-8192"
708                 messages=[{"role": "user", "content": prompt}],
709                 temperature=0.1, # low = factual, less hallucination
710                 max_tokens=1000
711             )
712             return response.choices[0].message.content
713         except Exception as e:
714             if attempt == 2:
715                 raise
716             time.sleep(2 ** attempt) # exponential backoff: 1s, 2s, 4s
717
718 H. HALLUCINATION PREVENTION CHECKLIST

```

-
- Prompt says "answer using ONLY the context provided"
 - Prompt says "do not invent code or behavior"
 - Temperature is set to 0.1 (not 0.7 or 1.0)
 - Similarity threshold filter removes low-quality chunks (< 0.4 score)
 - If no chunks pass the threshold, return "cannot find" directly (no LLM call)
 - Prompt instructs LLM to admit ignorance rather than guess

I. MULTI-FILE REASONING

When top_k=5, the retrieved chunks may come from 3-5 different files.
All of them are passed as context to the LLM simultaneously.
The LLM can then reason: "AuthMiddleware in auth.py calls UserRepository
in repositories/user.py which queries the users table."
This is why setting top_k=5 (not 1) is important for architectural questions.

J. COMMON RAG MISTAKES

- × Using a high temperature (0.7+) – causes creative hallucinations
 - Use 0.1 for factual retrieval tasks
- × Not filtering by similarity threshold
 - Irrelevant chunks confuse the LLM and reduce answer quality
- × Making the prompt too long
 - If context blocks exceed 6,000 tokens, the LLM may ignore later chunks
 - Limit each chunk to ~200 lines or truncate to 512 tokens
- × Using the same embedding model everywhere except query time
 - The query MUST be embedded with the exact same model used at index time
- × Not testing the "no relevant context" case
 - Always ask a completely off-topic question to verify graceful fallback

K. VALIDATION CHECKLIST

- Asking "how does login work?" returns code-based answer with file citations
- Asking "what is the capital of France?" returns "cannot find information"
- Answer contains [FILE: ...] markers matched to real file paths in the repo
- Temperature 0.1 is confirmed in Groq API call
- Retries work: temporarily set API key to invalid, verify retry logic fires

SECTION 7: DATABASE / VECTOR STORE LAYER

A. PURPOSE

767

768 ChromaDB stores the embeddings (vectors) and metadata for all code chunks.

769 When you search, ChromaDB compares your query vector against all stored

770 vectors and returns the most similar ones (by cosine similarity).

771 Think of it as a search engine where the search terms are mathematical

772 vectors instead of words.

773

774 B. RESPONSIBILITIES

775

- 776 - Store code chunk text, its vector embedding, and its metadata
- 777 - Support efficient similarity search across thousands of chunks
- 778 - Persist data to disk so re-embedding is not needed on restart
- 779 - Support collection-level namespacing (one collection per repo)
- 780 - Support deletion of collections (for re-indexing)

781

782 C. CORE FILE: core/vector_store.py

783

```
784 import chromadb
785 from chromadb.config import Settings as ChromaSettings
786 from config.settings import settings
787
788 _client = None
789
790 def get_client():
791     global _client
792     if _client is None:
793         _client = chromadb.PersistentClient(path=settings.chroma_persist_dir)
794     return _client
795
796 def get_or_create_collection(repo_name: str):
797     client = get_client()
798     # Sanitize collection name (ChromaDB requires alphanumeric + underscore)
799     safe_name = repo_name.replace("/", "__").replace("-", "_")
800     return client.get_or_create_collection(
801         name=safe_name,
802         metadata={"hnsw:space": "cosine"} # use cosine similarity
803     )
804
805 def upsert_chunks(repo_name: str, chunks: list):
806     collection = get_or_create_collection(repo_name)
807     ids = [f'{c["file_path"]}::{c["chunk_name"]}::{c["start_line"]}' for c in chunks]
808     embeddings = [c["embedding"] for c in chunks]
809     documents = [c["text"] for c in chunks]
810     metadatas = [{
811         "file_path": c["file_path"],
812         "chunk_type": c["chunk_type"],
813         "chunk_name": c["chunk_name"],
814         "start_line": c["start_line"],
```

```

815         "end_line":    c["end_line"],
816         "language":    c["language"],
817     } for c in chunks]
818     collection.upsert(ids=ids, embeddings=embeddings, documents=documents, metadatas=metadatas)
819
820 def similarity_search(repo_name: str, query_vector: list, top_k: int = 5) -> list:
821     collection = get_or_create_collection(repo_name)
822     results = collection.query(
823         query_embeddings=[query_vector],
824         n_results=top_k,
825         include=["documents", "metadatas", "distances"]
826     )
827     output = []
828     for i in range(len(results["documents"][0])):
829         output.append({
830             "text":    results["documents"][0][i],
831             "metadata": results["metadatas"][0][i],
832             "score":    1 - results["distances"][0][i] # convert distance to similarity
833         })
834     return output
835
836 def delete_collection(repo_name: str):
837     client = get_client()
838     safe_name = repo_name.replace("/", "__").replace("-", "_")
839     client.delete_collection(safe_name)
840
841 D. IMPORTANT: COSINE SIMILARITY vs DISTANCE
842 

---


843 ChromaDB with cosine space returns DISTANCE (lower = more similar).
844 Your THRESHOLD is a SIMILARITY score (higher = more similar).
845 Convert: similarity = 1 - distance
846 Always filter using: chunk["score"] >= threshold (e.g., >= 0.4)
847
848 E. COLLECTION NAMING STRATEGY
849 

---


850 One collection = one repository.
851 Collection name = owner__reponame (double underscore, sanitized)
852 Example: "tiangolo/fastapi" → "tiangolo__fastapi"
853
854 ChromaDB collection name rules:
855     - 3-63 characters
856     - alphanumeric and underscores only
857     - no hyphens (replace with underscores)
858     - no forward slashes
859
860 F. PERSISTENCE
861 

---


862 Using PersistentClient means data is saved to disk at chroma_persist_dir.

```

863 Default: ./chroma_db/
864 On restart, the data is still there – no need to re-embed.
865 Only re-embed when repository code actually changes.

866 867 G. COMMON VECTOR DB MISTAKES

- 868
869 × Using InMemoryClient instead of PersistentClient
 - 870 → Data lost on every restart; you'd need to re-embed on every startup
- 871
872 × Using Euclidean distance threshold instead of cosine similarity threshold
 - 873 → Different scale; always specify "hnsw:space": "cosine" and convert
- 874
875 × Using the same collection for multiple repos
 - 876 → Always namespace by repo name
- 877
878 × Not sanitizing repo names for ChromaDB's naming rules
 - 879 → Will crash with confusing errors; sanitize immediately

880 881 H. VALIDATION CHECKLIST

- 882
883 ☐ After upsert, call similarity_search and verify you get results back
- 884 ☐ Verify scores are between 0.0 and 1.0 (not negative or above 1)
- 885 ☐ Delete a collection, restart, confirm the collection is gone from disk
- 886 ☐ Upsert the same chunk twice (same ID) – verify it doesn't duplicate
- 887 ☐ Check the chroma_db/ directory exists on disk after first upsert

888
889
890 =====

891 SECTION 8: INGESTION PIPELINE IMPLEMENTATION

892 =====

893 894 A. PURPOSE

895
896 The ingestion pipeline is the "learning" phase of GitBrain.
897 It reads a GitHub repository and converts it into a searchable vector index.
898 This runs once per repository (and again when code changes).

899 900 B. RESPONSIBILITIES

- 901
902 - Fetch the file tree from GitHub (list of all files)
- 903 - Filter out non-code files (images, binaries, lock files)
- 904 - Fetch raw content for each code file
- 905 - Parse each file into meaningful code chunks
- 906 - Generate embeddings for each chunk
- 907 - Store everything in ChromaDB

908 909 C. CORE FILE: core/github_ingester.py

910

```
911 This file is the entry point for ingestion.
912 It coordinates all the steps above.
913
914 D. STEP 1: FETCH THE FILE TREE
915 

---


916 import requests
917
918 GITHUB_API = "https://api.github.com"
919 SKIP_EXTENSIONS = {".png", ".jpg", ".jpeg", ".gif", ".svg", ".ico",
920                   ".pdf", ".zip", ".tar", ".gz", ".exe", ".bin",
921                   ".lock", ".min.js", ".map"}
922 SKIP_DIRS = {"node_modules", ".git", "__pycache__", "dist", "build", "vendor"}
923
924 def get_headers(token=""):
925     headers = {"Accept": "application/vnd.github+json"}
926     if token:
927         headers["Authorization"] = f"Bearer {token}"
928     return headers
929
930 def fetch_file_tree(owner: str, repo: str, token: str = "") -> list:
931     # Get the default branch SHA
932     repo_info = requests.get(f"{GITHUB_API}/repos/{owner}/{repo}",
933                             headers=get_headers(token)).json()
934     default_branch = repo_info.get("default_branch", "main")
935     branch_info = requests.get(
936         f"{GITHUB_API}/repos/{owner}/{repo}/branches/{default_branch}",
937         headers=get_headers(token)).json()
938     sha = branch_info["commit"]["sha"]
939
940     # Get all files recursively
941     tree = requests.get(
942         f"{GITHUB_API}/repos/{owner}/{repo}/git/trees/{sha}?recursive=1",
943         headers=get_headers(token)).json()
944
945     files = []
946     for item in tree.get("tree", []):
947         if item["type"] != "blob":
948             continue
949         path = item["path"]
950         # Filter out binary/unwanted files
951         if any(path.endswith(ext) for ext in SKIP_EXTENSIONS):
952             continue
953         if any(d in path.split("/") for d in SKIP_DIRS):
954             continue
955         files.append({"path": path, "sha": item["sha"]})
956     return files
957
958 E. STEP 2: FETCH FILE CONTENTS WITH RATE LIMIT HANDLING
```

```

import base64, time

def fetch_file_content(owner: str, repo: str, path: str, token: str = "") -> str:
    response = requests.get(
        f"{GITHUB_API}/repos/{owner}/{repo}/contents/{path}",
        headers=get_headers(token)
    )
    # Check rate limit
    remaining = int(response.headers.get("X-RateLimit-Remaining", 100))
    if remaining < 10:
        reset_time = int(response.headers.get("X-RateLimit-Reset", time.time() + 60))
        wait = max(0, reset_time - time.time()) + 5
        print(f"Rate limit low ({remaining} remaining). Waiting {wait:.0f}s...")
        time.sleep(wait)

    if response.status_code == 200:
        data = response.json()
        if data.get("encoding") == "base64":
            return base64.b64decode(data["content"]).decode("utf-8", errors="replace")
    return ""

```

F. STEP 3: DETECT LANGUAGE FROM FILE PATH

```

def detect_language(path: str) -> str:
    ext = path.rsplit(".", 1)[-1].lower() if "." in path else ""
    return {
        "py": "python", "js": "javascript", "ts": "typescript",
        "java": "java", "go": "go", "rb": "ruby", "cpp": "cpp",
        "c": "c", "cs": "csharp", "rs": "rust", "md": "markdown",
        "yaml": "yaml", "yml": "yaml", "json": "json", "sh": "bash"
    }.get(ext, "text")

```

G. MAIN INGESTION ORCHESTRATOR

```

def ingest_repository(repo_url: str, token: str = "", incremental: bool = False) -> int:
    # Parse owner and repo from URL
    parts = repo_url.rstrip("/").split("/")
    owner, repo = parts[-2], parts[-1]
    repo_name = f"{owner}__{repo}"

    # Fetch file tree
    files = fetch_file_tree(owner, repo, token)
    print(f"Found {len(files)} files to process")

    all_chunks = []
    for file_info in files:
        content = fetch_file_content(owner, repo, file_info["path"], token)

```

```

1007         if not content.strip():
1008             continue
1009         language = detect_language(file_info["path"])
1010         chunks = code_chunker.chunk_file(
1011             content=content,
1012             file_path=file_info["path"],
1013             language=language
1014         )
1015         all_chunks.extend(chunks)
1016
1017     print(f"Generated {len(all_chunks)} chunks, embedding now...")
1018
1019     # Embed in batches of 32
1020     BATCH_SIZE = 32
1021     for i in range(0, len(all_chunks), BATCH_SIZE):
1022         batch = all_chunks[i:i+BATCH_SIZE]
1023         texts = [c["text"] for c in batch]
1024         embeddings = embedder.embed_batch(texts)
1025         for chunk, emb in zip(batch, embeddings):
1026             chunk["embedding"] = emb
1027
1028     # Upsert into ChromaDB
1029     vector_store.upsert_chunks(repo_name, all_chunks)
1030     return len(all_chunks)
1031

```

1032 H. COMMON INGESTION MISTAKES

- 1033
- 1034 × Fetching file content one by one without rate limit awareness
- 1035 → Add the rate limit check on every request
- 1036
- 1037 × Trying to decode binary files as UTF-8 without error handling
- 1038 → Always use errors="replace" or errors="ignore" in .decode()
- 1039
- 1040 × Not filtering node_modules or .git directories
- 1041 → Can result in 10,000+ irrelevant files being processed
- 1042
- 1043 × Not batching embedding calls
- 1044 → Embedding one chunk at a time is extremely slow
- 1045

1046 I. VALIDATION CHECKLIST

- 1047
- 1048 □ Running ingest on a 20-file repo produces 50-200 chunks
- 1049 □ No binary content appears in chunk text (no garbled characters)
- 1050 □ Each chunk has file_path, start_line, end_line, chunk_type in metadata
- 1051 □ ChromaDB collection is created and contains the expected number of entries
- 1052 □ Re-running ingest does NOT duplicate chunks (upsert by ID)
- 1053
- 1054

```
1055 =====
1056 SECTION 9: API COMMUNICATION FLOW
1057 =====
1058
1059 HOW DATA FLOWS BETWEEN ALL COMPONENTS
1060 _____
1061
1062 INGESTION REQUEST FLOW:
1063     User (Streamlit sidebar)
1064         → POST /ingest { repo_url, token, incremental }
1065         → FastAPI validates with Pydantic
1066         → github_ingester.ingest_repository()
1067             → fetch_file_tree() [GitHub API]
1068             → for each file: fetch_file_content() [GitHub API]
1069             → code_chunker.chunk_file() → Chunk objects
1070             → embedder.embed_batch() → vectors added
1071             → vector_store.upsert_chunks() → saved to ChromaDB
1072         → returns chunk count
1073         → FastAPI returns { status: "success", chunks_indexed: N }
1074         → Streamlit displays "Indexed N chunks"
1075
1076 QUERY REQUEST FLOW:
1077     User (Streamlit chat input)
1078         → POST /query { question, repo_name, top_k }
1079         → FastAPI validates with Pydantic
1080         → rag_engine.query()
1081             → embedder.embed_single(question) → query vector
1082             → vector_store.similarity_search() → top-5 chunks
1083             → filter by similarity threshold (>= 0.4)
1084             → build_prompt(question, chunks) → prompt string
1085             → llm_client.generate(prompt) → LLM answer text
1086             → parse_citations(answer, chunks) → sources list
1087         → returns { answer, sources }
1088         → FastAPI returns { answer, sources }
1089         → Streamlit renders answer bubble + citations expander
1090
1091 GITHUB ACTIONS RE-INDEX FLOW:
1092     Developer merges PR to main branch
1093         → GitHub Actions workflow triggers
1094         → POST /ingest { repo_url, incremental: true }
1095         → (same ingestion flow, but only changed files)
1096         → Workflow logs success or failure
1097
1098 IMPORTANT TIMING NOTES:
1099     - /ingest: can take 2-15 minutes for large repos. Use async or background tasks.
1100     - /query: should complete in 2-5 seconds (embed + search + LLM)
1101     - Frontend timeout for /ingest should be 300+ seconds
1102     - Frontend timeout for /query should be 30 seconds
```

BACKGROUND TASK PATTERN (for /ingest):

Instead of making the client wait 10 minutes, use FastAPI BackgroundTasks:

```
from fastapi import BackgroundTasks
```

```
@router.post("/ingest")
```

```
async def ingest(request: IngestRequest, background_tasks: BackgroundTasks):
```

```
    background_tasks.add_task(run_ingestion, request.repo_url, request.token)
```

```
    return {"status": "ingestion_started", "message": "Check /status for progress"}
```

Then add a GET /status endpoint that checks a shared status dict

```
ingestion_status = {} # { repo_name: { status, chunks_indexed, error } }
```

SECTION 10: AUTH / SECURITY IMPLEMENTATION

NOTE: Full authentication (OAuth, user accounts) is explicitly OUT of MVP scope.
The following is what you DO need for the MVP:

A. GITHUB TOKEN (required for private repos and higher rate limits)

- Token goes in .env file as GITHUB_TOKEN=ghp_XXXXX
- Never hardcode it in Python files
- Frontend passes it to /ingest as an optional field
- Backend reads it from request body OR falls back to settings.github_token
- NEVER log the token value

B. GROQ API KEY

- Goes in .env as GROQ_API_KEY=gsk_XXXXX
- Loaded via settings.groq_api_key
- Passed to Groq() client at initialization
- Never returned in any API response

C. PROTECTING YOUR .env FILE

- Add .env to .gitignore on Day 1 (before first commit)
- Provide .env.example with placeholder values
- Double-check with: `git status` - .env should never appear

D. SIMPLE API KEY FOR YOUR OWN BACKEND (optional)

If you want to protect your /ingest and /query endpoints from public abuse when deployed, add a simple API key check:


```

1151     from fastapi import Security, HTTPException
1152     from fastapi.security import APIKeyHeader
1153
1154     api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)
1155
1156     def verify_api_key(api_key: str = Security(api_key_header)):
1157         if api_key != settings.gitbrain_api_key:
1158             raise HTTPException(status_code=403, detail="Invalid API key")
1159         return api_key
1160
1161     # Add to endpoint: async def ingest(request: IngestRequest, _=Depends(verify_api_key))
1162
1163     E. VALIDATION CHECKLIST
1164     _____
1165     □ .env is in .gitignore and does NOT appear in git status
1166     □ .env.example exists with placeholder values
1167     □ No API keys or tokens appear in any log output
1168     □ The GitHub token is correctly loaded and rate limit is 5000/hr (not 60/hr)
1169
1170
1171     =====
1172     SECTION 11: TESTING IMPLEMENTATION GUIDE
1173     =====
1174
1175     TESTING PHILOSOPHY FOR THIS PROJECT
1176     _____
1177
1178     Don't aim for 100% coverage on your first pass.
1179     Aim for:
1180         - Tests that catch the bugs YOU are most likely to make
1181         - Tests that verify your RAG pipeline actually works
1182         - Tests that prove your ingestion produces correct metadata
1183
1184     Run tests before every major integration step.
1185
1186     A. UNIT TEST: code_chunker.py
1187     _____
1188
1189     # tests/test_chunker.py
1190
1191     SAMPLE_PYTHON = '''
1192     def add(a, b):
1193         """Add two numbers."""
1194         return a + b
1195
1196     class Calculator:
1197         def multiply(self, x, y):
1198             return x * y
1199
1200     '''
1201
1202     def test_python_chunker_finds_function():

```

```

1199     chunks = chunk_file(SAMPLE_PYTHON, "math.py", "python")
1200     chunk_names = [c["chunk_name"] for c in chunks]
1201     assert "add" in chunk_names
1202
1203 def test_python_chunker_finds_class():
1204     chunks = chunk_file(SAMPLE_PYTHON, "math.py", "python")
1205     chunk_names = [c["chunk_name"] for c in chunks]
1206     assert "Calculator" in chunk_names
1207
1208 def test_chunk_has_required_metadata():
1209     chunks = chunk_file(SAMPLE_PYTHON, "math.py", "python")
1210     for chunk in chunks:
1211         assert "file_path" in chunk
1212         assert "start_line" in chunk
1213         assert "end_line" in chunk
1214         assert "chunk_type" in chunk
1215         assert "text" in chunk
1216
1217 def test_start_line_less_than_end_line():
1218     chunks = chunk_file(SAMPLE_PYTHON, "math.py", "python")
1219     for chunk in chunks:
1220         assert chunk["start_line"] <= chunk["end_line"]
1221
1222 B. UNIT TEST: embedder.py
1223 

---


1224 # tests/test_embedder.py
1225
1226 def test_embedding_dimension():
1227     vectors = embed_batch(["hello world"])
1228     assert len(vectors[0]) == 384 # all-MiniLM-L6-v2 output size
1229
1230 def test_same_text_produces_same_embedding():
1231     v1 = embed_batch(["test text"])[0]
1232     v2 = embed_batch(["test text"])[0]
1233     assert v1 == v2 # deterministic model
1234
1235 def test_different_texts_produce_different_embeddings():
1236     v1 = embed_batch(["python function"])[0]
1237     v2 = embed_batch(["database schema"])[0]
1238     assert v1 != v2
1239
1240 C. UNIT TEST: vector_store.py
1241 

---


1242 # tests/test_vector_store.py
1243 import chromadb, pytest
1244
1245 @pytest.fixture
1246 def temp_collection():

```

```

1247     client = chromadb.EphemeralClient()
1248     collection = client.create_collection("test_repo", metadata={"hnsw:space": "cosine"})
1249     yield collection
1250     client.delete_collection("test_repo")
1251
1252 def test_upsert_and_query(temp_collection):
1253     # Upsert a chunk
1254     temp_collection.upsert(
1255         ids=["chunk_1"],
1256         embeddings=[[0.1]*384],
1257         documents=["def login(): pass"],
1258         metadatas=[{"file_path": "auth.py", "start_line": 1, "end_line": 2}]
1259     )
1260     # Query
1261     results = temp_collection.query(query_embeddings=[[0.1]*384], n_results=1)
1262     assert len(results["documents"][0]) == 1
1263     assert "login" in results["documents"][0][0]
1264
1265 D. INTEGRATION TEST: END-TO-END INGESTION
1266 

---


1267 # tests/test_integration.py
1268 # Use a tiny test repo you control or a mock
1269
1270 def test_ingest_small_repo():
1271     # Create a temp directory with 3 Python files
1272     # Run ingest on it
1273     # Verify ChromaDB has entries
1274     count = ingest_repository("https://github.com/your-test-org/tiny-test-repo")
1275     assert count > 0
1276
1277 E. INTEGRATION TEST: FASTAPI ENDPOINTS
1278 

---


1279 from fastapi.testclient import TestClient
1280 from api.main import app
1281
1282 client = TestClient(app)
1283
1284 def test_health_endpoint():
1285     response = client.get("/health")
1286     assert response.status_code == 200
1287     assert response.json()["status"] == "ok"
1288
1289 def test_query_unindexed_repo_returns_error():
1290     response = client.post("/query", json={
1291         "question": "how does it work?",
1292         "repo_name": "nonexistent__repo"
1293     })
1294     assert response.status_code in [404, 200] # 200 with "cannot find" is also ok

```

F. MANUAL ACCEPTANCE TEST SCRIPT

Run these manually after completing each phase:

Phase 1 done?

```
python -c "from core.github_ingester import fetch_file_tree; print(fetch_file_tree('tiangolo', 'fastapi')[:3])"
```

→ Should print 3 file paths

Phase 2 done?

```
python -c "from core.embedder import embed_batch; v=embed_batch(['test']); print(len(v[0]))"
```

→ Should print 384

Phase 3 done?

```
curl -X POST localhost:8000/query -H "Content-Type: application/json" -d '{"question": "what is this?", "repo_name": "tiangolo__fastapi"}'
```

→ Should return JSON with answer field

Phase 4 done?

□ Open Streamlit in browser, ask 5 questions, verify answers make sense

G. AI RESPONSE VALIDATION TEST

Keep a file: tests/test_questions.json

Format:

```
[
  {
    "question": "How is authentication handled?",
    "expected_file_keywords": ["auth", "middleware", "token"],
    "should_cite": true
  },
  {
    "question": "What is the capital of France?",
    "expected_answer_contains": "cannot find information",
    "should_cite": false
  }
]
```

Write a script that runs these and checks responses automatically.

H. VALIDATION CHECKLIST

- All pytest unit tests pass: `pytest tests/ -v`
- FastAPI TestClient integration tests pass
- Asking a relevant question returns a cited answer
- Asking an off-topic question returns "cannot find information"
- Ingesting the same repo twice doesn't double the chunk count
- Re-indexing after a file change makes the new code discoverable

```
1341
1342
1343 =====
1344 SECTION 12: DEVOPS / DEPLOYMENT GUIDE
1345 =====
1346
1347 A. LOCAL DEVELOPMENT SETUP (do this first)
1348 

---


1349 Step 1: Clone your repo
1350     git clone https://github.com/yourusername/gitbrain.git
1351     cd gitbrain
1352
1353 Step 2: Create virtual environment
1354     python -m venv venv
1355     source venv/bin/activate          # Linux/Mac
1356     venv\Scripts\activate            # Windows
1357
1358 Step 3: Install dependencies
1359     pip install fastapi uvicorn[standard] chromadb sentence-transformers \
1360         groq streamlit requests python-dotenv pydantic-settings pytest
1361
1362 Step 4: Create .env file
1363     cp .env.example .env
1364     # Edit .env and fill in your tokens
1365
1366 Step 5: Start backend
1367     uvicorn api.main:app --reload --port 8000
1368
1369 Step 6: Start frontend (separate terminal)
1370     streamlit run frontend/app.py
1371
1372 Step 7: Open browser
1373     http://localhost:8501
1374
1375 B. REQUIREMENTS.TXT
1376 

---


1377 Keep this file updated. Pin major versions for reproducibility.
1378 fastapi==0.111.0
1379 uvicorn[standard]==0.30.0
1380 chromadb==0.5.0
1381 sentence-transformers==3.0.0
1382 groq==0.9.0
1383 streamlit==1.36.0
1384 requests==2.32.0
1385 python-dotenv==1.0.1
1386 pydantic-settings==2.3.0
1387 pytest==8.2.0
1388
```

C. GITHUB ACTIONS REINDEX WORKFLOW

Place in: `.github/workflows/reindex.yml` (inside the TARGET repository)

```
name: GitBrain Re-Index
```

```
on:
```

```
  pull_request:
```

```
    types: [closed]
```

```
    branches: [main]
```

```
jobs:
```

```
  reindex:
```

```
    if: github.event.pull_request.merged == true
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Trigger GitBrain Re-Indexing
```

```
        run: |
```

```
          curl -X POST "${{ secrets.GITBRAIN_API_URL }}/ingest" \
```

```
            -H "Content-Type: application/json" \
```

```
            -H "X-API-Key: ${{ secrets.GITBRAIN_API_KEY }}" \
```

```
            -d '{
```

```
              "repo_url": "https://github.com/${{ github.repository }}",
```

```
              "incremental": true
```

```
            }'
```

GitHub repo secrets to set:

```
GITBRAIN_API_URL = https://your-railway-app.up.railway.app
```

```
GITBRAIN_API_KEY = your-optional-api-key
```

D. DEPLOY FASTAPI TO RAILWAY (free tier)

Step 1: Push your code to GitHub

Step 2: Go to railway.app and sign up with GitHub

Step 3: "New Project" → "Deploy from GitHub Repo" → select your repo

Step 4: Railway auto-detects FastAPI – set start command:

```
uvicorn api.main:app --host 0.0.0.0 --port $PORT
```

Step 5: Go to Variables tab → add all `.env` variables

Step 6: Deploy – Railway gives you a URL like: <https://gitbrain-xxx.up.railway.app>

NOTE: Railway's free tier has ephemeral storage.

The ChromaDB data (`chroma_db/`) is lost on each redeploy.

For demos, this is acceptable – just re-ingest after each deployment.

E. DEPLOY STREAMLIT TO STREAMLIT COMMUNITY CLOUD

Step 1: Push `frontend/app.py` to GitHub

Step 2: Go to share.streamlit.io and sign up

Step 3: "New app" → connect your GitHub repo → set main file: frontend/app.py

Step 4: In "Secrets" section, add:

```
BACKEND_URL = "https://your-railway-app.up.railway.app"
```

Step 5: Deploy – Streamlit gives you a public URL

In frontend/app.py, read the backend URL:

```
import streamlit as st
```

```
BACKEND_URL = st.secrets.get("BACKEND_URL", "http://localhost:8000")
```

F. DEMO WITH NGROK (easiest for local demos)

If you want to demo without deploying:

Step 1: Run backend locally: `uvicorn api.main:app --port 8000`

Step 2: Run ngrok: `ngrok http 8000`

Step 3: Copy the ngrok URL (e.g., `https://abc123.ngrok.io`)

Step 4: Use this URL as `GITBRAIN_API_URL` in GitHub Actions secrets

Step 5: Run Streamlit locally and change `BACKEND_URL` to the ngrok URL

G. ENVIRONMENT VARIABLE CHECKLIST

Local `.env`:

- `GITHUB_TOKEN=ghp_...`

- `GROQ_API_KEY=gsk_...`

- `CHROMA_PERSIST_DIR=./chroma_db`

- `EMBEDDING_MODEL=all-MiniLM-L6-v2`

- `LLM_MODEL=llama3-8b-8192`

- `TOP_K_RESULTS=5`

- `SIMILARITY_THRESHOLD=0.4`

Railway Variables (same as above, set in dashboard):

- All of the above, PLUS `PORT` is set automatically by Railway

Streamlit Secrets:

- `BACKEND_URL=https://your-railway-app.up.railway.app`

GitHub Actions Secrets (in target repo):

- `GITBRAIN_API_URL`

- `GITBRAIN_API_KEY` (optional)

SECTION 13: COMMON BEGINNER MISTAKES

MISTAKE 1: Chunking entire files as single units

Problem: A 500-line file as one chunk is too big for retrieval.

The LLM gets confused by too much irrelevant context.

1485 Fix: Always use AST-based or heuristic chunking.
1486 A chunk should be 10-80 lines. If it's over 150 lines, split it further.
1487
1488 MISTAKE 2: Using different embedding models for index and query
1489

1490 Problem: Index uses model A, query uses model B.
1491 Vectors are in completely different spaces – retrieval fails silently.
1492 Fix: Define `embedding_model` in `settings.py`.
1493 Import `settings.embedding_model` in BOTH `embedder.py` calls.
1494 Test: embed "hello" at index time and query time, compare vectors.
1495
1496 MISTAKE 3: Building the UI before the pipeline works
1497

1498 Problem: You waste hours debugging UI when the real bug is in the RAG pipeline.
1499 Fix: Test the complete pipeline with curl FIRST.
1500 Only start the UI after curl /query returns a correct answer.
1501
1502 MISTAKE 4: Not filtering by similarity threshold
1503

1504 Problem: Chunks with 0.2 similarity score are garbage.
1505 Including them in the prompt actively hurts answer quality.
1506 Fix: Filter results: `[r for r in results if r["score"] >= 0.4]`
1507 If no results pass the threshold, return "cannot find information" directly.
1508
1509 MISTAKE 5: Forgetting to decode GitHub base64 content
1510

1511 Problem: GitHub API returns file content as base64.
1512 Using it raw gives you garbled binary-looking text.
1513 Fix: Always: `base64.b64decode(data["content"]).decode("utf-8", errors="replace")`
1514
1515 MISTAKE 6: Re-embedding everything on every restart
1516

1517 Problem: First-time embedding takes 5 minutes for a medium repo.
1518 If you re-embed on every restart, development becomes unbearably slow.
1519 Fix: Use ChromaDB PersistentClient.
1520 Check if collection already exists and has data before re-embedding.
1521
1522 MISTAKE 7: Blocking the HTTP response during ingestion
1523

1524 Problem: POST /ingest takes 10 minutes. Browser gives up after 30 seconds.
1525 Fix: Use FastAPI BackgroundTasks.
1526 Return immediately with "ingestion started" and poll /status for progress.
1527
1528 MISTAKE 8: Setting LLM temperature too high
1529

1530 Problem: `temperature=0.7` means the LLM is "creative" – it invents code.
1531 Fix: Use `temperature=0.1` for factual code explanation tasks.
1532 Only increase temperature for open-ended creative questions.

1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579

MISTAKE 9: Printing tokens or API keys in logs

Problem: ``logger.debug(f"Using token: {token}")`` leaks credentials.
Fix: Log token presence only: ``logger.debug(f"Token provided: {bool(token)}")``

MISTAKE 10: Not handling ChromaDB collection name constraints

Problem: "tiangolo/fastapi" has a slash – ChromaDB rejects it as a name.
Fix: Always sanitize: `repo_name.replace("/", "__").replace("-", "_")`
Test: try creating a collection with a slash – confirm the error you get.

MISTAKE 11: Committing .env to Git

Problem: GitHub shows your API keys publicly. Account gets compromised.
Fix: Add .env to .gitignore on DAY 1.
Check: `git ls-files | grep ".env"` – this should return nothing.

MISTAKE 12: Not handling GitHub binary files

Problem: Trying to decode a PNG image as UTF-8 gives thousands of garbled characters that pollute your vector store.
Fix: Skip files with binary extensions BEFORE fetching content.
Also skip files larger than 1MB (GitHub truncates them anyway).

=====

SECTION 14: DEBUGGING CHECKLIST

=====

SYMPTOM: Ingestion seems to run but ChromaDB has 0 chunks

- ❑ Is the GitHub token valid? (test: `curl -H "Authorization: Bearer TOKEN" https://api.github.com/user`)
- ❑ Is the repo public, or is the token scoped for private repos?
- ❑ Are ALL files being filtered out? (print the file list before chunking)
- ❑ Is the chunker returning empty lists? (print chunk count per file)
- ❑ Is the ChromaDB path correct and writable?

SYMPTOM: Query returns no results / empty sources

- ❑ Is the collection name the same at index and query time?
- ❑ Are you using the SAME embedding model for both index and query?
- ❑ Are the similarity scores all below 0.4? (temporarily lower threshold to 0.1)
- ❑ Does the ChromaDB collection actually have data? (check `collection.count()`)

SYMPTOM: LLM answer is wrong / hallucinating

- 1580 □ Print the FULL prompt sent to the LLM – are the chunks relevant?
- 1581 □ Are the similarity scores actually high (>0.6)? If low, retrieval is the problem
- 1582 □ Is temperature set to 0.1?
- 1583 □ Is the system prompt including "answer ONLY from context"?

1584
1585 SYMPTOM: Streamlit chat shows old answers after asking new questions
1586 _____

- 1587 □ Are you calling `st.rerun()` after appending to `session_state.messages`?
- 1588 □ Did you append the user message AND the assistant message before `rerun`?

1589
1590 SYMPTOM: ChromaDB collection names causing crashes
1591 _____

- 1592 □ Does the `repo_name` contain slashes, hyphens, or spaces?
- 1593 □ Is the sanitization function applied before EVERY ChromaDB call?

1594
1595 SYMPTOM: GitHub Actions workflow fires but ingestion fails
1596 _____

- 1597 □ Is `GITBRAIN_API_URL` set in the repo's Secrets (not variables)?
- 1598 □ Is the FastAPI backend running and publicly accessible?
- 1599 □ Does the `/ingest` endpoint require an API key that Actions isn't sending?
- 1600 □ Check Actions logs – the `curl` command output will show the HTTP response

1601
1602 SYMPTOM: Slow ingestion (30+ minutes for a medium repo)
1603 _____

- 1604 □ Are you hitting GitHub rate limits? (check remaining count in headers)
- 1605 □ Are you embedding one chunk at a time instead of in batches?
- 1606 □ Is the embedding model being reloaded for every batch? (load once, reuse)

1607
1608 GENERAL DEBUG TOOLS
1609 _____

1610 Test ChromaDB directly:

```
1611         python -c "import chromadb; c=chromadb.PersistentClient('./chroma_db'); print(c.list_collections())"
```

1612
1613 Test GitHub token:

```
1614         curl -H "Authorization: Bearer YOUR_TOKEN" https://api.github.com/rate_limit
```

1615
1616 Test Groq API:

```
1617         python -c "from groq import Groq; c=Groq(); r=c.chat.completions.create(model='llama3-8b-8192', messages=[{'role':'user','content':'say hi'}],max_tokens=10); print(r.choices[0].message.content)"
```

1618
1619 Test embedding:

```
1620         python -c "from sentence_transformers import SentenceTransformer; m=SentenceTransformer('all-MiniLM-L6-v2'); print(len(m.encode('test')))"
```

1621
1622
1623 =====

1624 SECTION 15: MVP BUILD ORDER

1625 =====

1626
1627 If you're starting fresh right now, follow this exact order.
1628 Do NOT skip ahead. Each step depends on the previous one working.
1629

1630
1631 DAY 1-2: ENVIRONMENT + PROJECT STRUCTURE
1632

- 1633 1. Create project folder: gitbrain/
- 1634 2. Create virtual environment and activate it
- 1635 3. Create folder structure:
 - 1636 gitbrain/api/routes/, gitbrain/core/, gitbrain/config/,
 - 1637 gitbrain/frontend/, gitbrain/tests/
- 1638 4. Create .env.example with all variable names (empty values)
- 1639 5. Create .env with your real tokens
- 1640 6. Add .env to .gitignore
- 1641 7. git init, git add ., git commit -m "initial structure"
- 1642 8. Create config/settings.py with Pydantic BaseSettings

1643
1644 VERIFY: python -c "from config.settings import settings; print(settings.llm_model)"
1645

1646
1647 DAY 3-4: GITHUB INGESTION MODULE
1648

- 1649 9. Create core/github_ingester.py
- 1650 10. Implement fetch_file_tree() with rate limit handling
- 1651 11. Implement fetch_file_content() with base64 decoding
- 1652 12. Add file filtering (binary extensions, skip dirs)
- 1653 13. Test: python -c "from core.github_ingester import fetch_file_tree; print(len(fetch_file_tree('tiangolo','fastapi','')))"
1654

1655 VERIFY: Should print a number > 50 (number of files in the repo)
1656

1657
1658 DAY 5-6: CODE CHUNKING MODULE
1659

- 1660 14. Create core/code_chunker.py
- 1661 15. Implement AST-based Python chunker (functions + classes)
- 1662 16. Implement sliding window fallback for other file types
- 1663 17. Write test_chunker.py and run pytest tests/test_chunker.py
1664

1665 VERIFY: chunk_file() on a sample .py file returns list of dicts with
1666 file_path, chunk_name, start_line, end_line, text fields
1667

1668
1669 DAY 7: EMBEDDING MODULE
1670

- 1671 18. Create core/embedder.py

1672 19. Implement embed_batch() returning list of 384-dimensional vectors
1673 20. Implement embed_single() for single string queries
1674 21. Test: verify dimension is 384
1675
1676 VERIFY: embed_batch(["hello"]) returns [[float×384]]
1677
1678

1679 DAY 8: VECTOR STORE MODULE
1680

1681 22. Create core/vector_store.py
1682 23. Implement get_or_create_collection() with cosine distance
1683 24. Implement upsert_chunks() with proper ID generation
1684 25. Implement similarity_search() converting distance to similarity score
1685 26. Implement delete_collection()
1686
1687 VERIFY: Upsert 3 test chunks, query with similar text, get those chunks back
1688
1689

1690 DAY 9: CONNECT THE INGESTION PIPELINE (end-to-end test)
1691

1692 27. In github_ingester.py, add ingest_repository() orchestrator function
1693 28. Wire: fetch_file_tree → fetch each file → chunk → embed → upsert
1694 29. Test: python -c "from core.github_ingester import ingest_repository; print(ingest_repository
(['https://github.com/tiangolo/fastapi'](https://github.com/tiangolo/fastapi)))"

1695
1696 VERIFY: Should print a chunk count and ChromaDB should have data
1697 Check: python -c "import chromadb; c=chromadb.PersistentClient('./chroma_db'); col=c.get_c
ollection('tiangolo__fastapi'); print(col.count())"
1698
1699

1700 DAY 10-11: LLM CLIENT + RAG ENGINE
1701

1702 30. Create core/llm_client.py – wrap Groq API with retry logic
1703 31. Create core/rag_engine.py – implement query() function
1704 32. Test: python -c "from core.rag_engine import query; print(query('how does routing work?', 'tia
ngolo__fastapi'))"
1705
1706 VERIFY: Should print {"answer": "...", "sources": [...]} with real content
1707
1708

1709 DAY 12-13: FASTAPI BACKEND
1710

1711 33. Create api/main.py + api/routes/health.py, ingest.py, query.py
1712 34. Wire routes to core modules
1713 35. Start server: uvicorn api.main:app --reload --port 8000
1714 36. Test all 3 endpoints with curl
1715
1716 VERIFY: All curl tests from Section 4.K pass

1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764

DAY 14-15: STREAMLIT FRONTEND

- 37. Create frontend/app.py
- 38. Build sidebar with ingestion button
- 39. Build chat area with session state
- 40. Connect to backend via requests
- 41. Test full UI flow in browser

VERIFY: Can ingest a repo and get a cited answer entirely through the browser

WEEK 6: AUTOMATION + POLISH

- 42. Add incremental re-indexing (diff-based, only changed files)
- 43. Create .github/workflows/reindex.yml
- 44. Test GitHub Actions trigger on a PR merge in a test repo
- 45. Polish citation display in Streamlit
- 46. Add error states and loading indicators

WEEK 7-8: TESTING + DEPLOYMENT + DOCUMENTATION

- 47. Write and run all pytest tests
- 48. Deploy backend to Railway, frontend to Streamlit Cloud
- 49. Record a demo video showing end-to-end functionality
- 50. Write README with setup instructions, architecture diagram, and screenshots
- 51. Finalize internship report

=====

SECTION 16: FUTURE ENHANCEMENTS

=====

After completing the MVP, here are additions ordered by value vs effort:

HIGH VALUE, LOW EFFORT

- 1. Streaming responses
 - Use Groq's streaming API and FastAPI StreamingResponse.
 - The UI feels much faster even if total time is the same.
 - Effort: 1-2 days
- 2. Conversation memory (follow-up questions)
 - Keep a rolling window of the last 3 question-answer pairs.
 - Include them in the prompt so users can ask "what about line 42 you mentioned?"
 - Effort: 1 day

3. File-level summary on click
When user clicks a file in the file tree sidebar,
auto-ask "summarize the purpose of {filename}" and show result.
Effort: 0.5 days (reuse query pipeline)

4. Query caching
Cache recent query results in a Python dict (in-memory).
Repeated questions return instantly without calling Groq.
Effort: 1 day

HIGH VALUE, MEDIUM EFFORT

5. Multi-repository support
Add a repository selector dropdown.
Each repo has its own ChromaDB collection (already implemented).
Effort: 2-3 days

6. Incremental re-indexing (diff-based)
Use GitHub commit API to find changed files since last index.
Only re-embed changed files, not the entire repo.
Effort: 2-3 days

7. Export answers as Markdown
Add a "Copy as Markdown" button below each answer.
Include the question, answer, and citations in a markdown block.
Effort: 0.5 days

8. GitHub Issues context
Index issue titles and descriptions alongside code.
Lets users ask "what issues are open about authentication?"
Effort: 2-3 days

ADVANCED (Phase 2+)

9. Private repo OAuth flow (GitHub App)
10. Redis caching layer for repeated queries
11. VS Code extension wrapping the /query API
12. Dependency graph for cross-module reasoning
13. Voice input via browser Web Speech API
14. Answer export as PDF
15. Multi-user support with auth (JWT/OAuth)

PRIORITIZATION RULE:

Always ask: "Does this help me demo GitBrain more impressively in 60 seconds?"
If yes → build it.
If no → defer it.

```
1813 =====
1814 END OF GITBRAIN IMPLEMENTATION BLUEPRINT
1815 -----
1816 Total sections: 16
1817 Total implementation steps in MVP build order: 51
1818 Estimated time for motivated beginner: 7-8 weeks
1819 Minimum viable demo time: 2-3 weeks (Sections 8 + 6 + 4 only)
1820
1821 QUICK REFERENCE LINKS:
1822 Groq (free LLM API):      https://console.groq.com
1823 ChromaDB docs:           https://docs.trychroma.com
1824 sentence-transformers:    https://www.sbert.net
1825 FastAPI docs:            https://fastapi.tiangolo.com
1826 Streamlit docs:          https://docs.streamlit.io
1827 GitHub REST API:         https://docs.github.com/en/rest
1828 Railway (free hosting):    https://railway.app
1829 Streamlit Community Cloud: https://share.streamlit.io
1830
1831 Good luck. Build it one vertical slice at a time.
1832 =====
1833
```